GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	**Course Code**	E2UC302B
Program	B.Tech. (AIML)	**Academic Year**	2026–27
Semester/Batch	III	**Lab Assignment No**	04

		o.	
Set	SETA	**Date**	
FacultyIn-Charge	Mr.ArunendraManiTripathi	**Max.Marks**	10
WeekNo.	5	**Duration**	90minutes

CourseOutcome(s)	CO3: Apply SQL techniques, including single-row functions and multi-table queries, to retrieve and reformat meaningful information from relational databases.

Skill(s) Addressed	SQL Programming, Single-Row Functions, Multi-table Queries, Salary & Department Analysis

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Execute all SQL queries in MySQL Workbench (or the prescribed SQL client).
- Use the dataset/schema given below and retain the same table and column names.
- Show both the SQL query and the corresponding output for every question.
- Save the SQL script as RollNo_Name_DBMS_Lab04_SETA.sql.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Use single-row (string, numeric and date) functions to reformat and analyse employee data, then combine Employee with the new Department table using multi-table (Cartesian product and old-style WHERE-based) queries.

3. DATASET / TABLE SCHEMA

Employee table (already created in Week 4 — re-create only if not already present):

```
CREATE TABLE Employee (
  Emp_ID     INT PRIMARY KEY,
  Emp_Name   VARCHAR(30),
  Department VARCHAR(20),
  Salary     DECIMAL(10,2),
  Experience INT
);

INSERT INTO Employee VALUES
(1,'Rahul','IT',55000,3),
(2,'Simran','IT',72000,5),
(3,'Farhan','HR',48000,2),
(4,'Divya','Sales',39000,2),
(5,'Vikram','IT',65000,4),
(6,'Meera','HR',61000,6),
(7,'Aman','Sales',45500,3),
(8,'Anjali','IT',83000,7),
(9,'Rohit','HR',52000,4),
(10,'Priya','Sales',47000,5);
```

New this week — extend the dataset with a date column and a related table:

```
-- Add a date column for this week's date-function tasks
ALTER TABLE Employee ADD COLUMN Joining_Date DATE;
UPDATE Employee SET Joining_Date='2022-03-14' WHERE Emp_ID=1;
UPDATE Employee SET Joining_Date='2021-07-01' WHERE Emp_ID=2;
```

```
UPDATE Employee SET Joining_Date='2023-01-20' WHERE Emp_ID=3;
UPDATE Employee SET Joining_Date='2020-11-05' WHERE Emp_ID=4;
UPDATE Employee SET Joining_Date='2022-09-10' WHERE Emp_ID=5;
UPDATE Employee SET Joining_Date='2019-06-18' WHERE Emp_ID=6;
UPDATE Employee SET Joining_Date='2023-04-02' WHERE Emp_ID=7;
UPDATE Employee SET Joining_Date='2018-12-30' WHERE Emp_ID=8;
UPDATE Employee SET Joining_Date='2021-02-14' WHERE Emp_ID=9;
UPDATE Employee SET Joining_Date='2020-08-22' WHERE Emp_ID=10;

-- New table for this week's multi-table / Cartesian product tasks
CREATE TABLE Department (
  Dept_Name VARCHAR(20) PRIMARY KEY,
  Location  VARCHAR(20),
  HOD       VARCHAR(30)
);

INSERT INTO Department VALUES
('IT','Bengaluru','Anjali Rao'),
('HR','Delhi','Vivek Anand'),
('Sales','Mumbai','Kabir Shah');
```

4. QUESTIONS / TASKS

Q1. Using string functions, display Emp_Name in UPPERCASE, its LENGTH, and the first three characters using SUBSTR, for every employee.

Q2. Using numeric functions, display Salary ROUNDed to the nearest thousand, TRUNCated to the nearest 1000, and the remainder of Salary MOD 1000, for every employee.

Q3. Using CONCAT, display a single column combining Emp_Name and Department formatted as 'Emp_Name (Department)'.

Q4. Using date functions on Joining_Date, display each employee's tenure in completed months using TIMESTAMPDIFF(MONTH, Joining_Date, CURDATE()), and the date one year after Joining_Date using DATE_ADD.

Q5. Write a query producing the CARTESIAN PRODUCT (CROSS JOIN) of Employee and Department. Note the row count returned and verify it equals (rows in Employee) x (rows in Department).

Q6. Write a multi-table query using the old-style (WHERE-based) join to combine Employee and Department, matching Employee.Department = Department.Dept_Name, displaying Emp_Name, Department, Location and HOD.

Q7. Extend the query from the previous task with an additional WHERE condition to display only employees whose department is located in 'Bengaluru'.

Q8. Compare the row counts from the Cartesian product and the filtered join, and explain in 2-3 lines why an unfiltered Cartesian product is rarely useful in practice.

Q9. Using a WHERE-based multi-table query, display each employee's name alongside their department's HOD, sorted by Department.

5. STUDY MATERIAL / REFERENCE

- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- String Functions: https://dev.mysql.com/doc/refman/8.0/en/string-functions.html
- Numeric / Mathematical Functions: https://dev.mysql.com/doc/refman/8.0/en/mathematical-functions.html
- Date and Time Functions: https://dev.mysql.com/doc/refman/8.0/en/date-and-time-functions.html
- Joins (Multi-table queries, Cartesian product): https://dev.mysql.com/doc/refman/8.0/en/join.html

6. EVALUATION RUBRIC

Evalua	Marks

tionCriteria	
Correctuseofstring,numericanddatefunctions	3
Corre	2

ctCROSSJOIN/Cartesianproductquery	
CorrectWHERE-basedmulti-tab	2

Ie join with filter condition	
Correct output interpretation and verif	1

ication	
SQL script quality and formatting	1
Viva/spot-check	1
TOTAL	**10**

7. EVALUATION PROCESS

1. Submission check — SQL script and outputs received within the deadline.
2. Execution check — queries are executed successfully and outputs are verified.
3. Rubric scoring — marks are awarded against the stated criteria.
4. Viva/spot-check — brief questions may be asked to verify individual understanding.

GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	**Course Code**	E2UC302B
Program	B.Tech. (AIML)	**Academic Year**	2026–27
Semester/Batch	III	**Lab Assignment No**	05

		o.	
Set	SETB	**Date**	
Faculty In-Charge	Mr. Arunendra Mani Tripathi	**Max. Marks**	10
Week No.	5	**Duration**	90 minutes

Course Outcome(s)	CO3: Apply SQL techniques, including single-row functions and multi-table queries, to retrieve and reformat meaningful information from relational databases.

Skill(s) Addressed	SQL Programming, Single-Row Functions, Multi-table Queries, Academic Data Analysis

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Execute all SQL queries in MySQL Workbench (or the prescribed SQL client).
- Use the dataset/schema given below and retain the same table and column names.
- Show both the SQL query and the corresponding output for every question.
- Save the SQL script as RollNo_Name_DBMS_Lab05_SETB.sql.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Use single-row (string, numeric and date) functions to reformat and analyse student data, then combine Student with the new Department table using multi-table (Cartesian product and old-style WHERE-based) queries.

3. DATASET / TABLE SCHEMA

Student table (already created in Week 4 — re-create only if not already present):

```
CREATE TABLE Student (
  Student_ID INT PRIMARY KEY,
  Name       VARCHAR(30),
  Department VARCHAR(20),
  Semester   INT,
  CGPA       DECIMAL(3,2),
  Attendance DECIMAL(5,2)
);

INSERT INTO Student VALUES
(101,'Rahul','AIML',4,8.40,88.5),
(102,'Anjali','AIML',4,9.10,92.0),
(103,'Amit','CSE',4,7.20,71.0),
(104,'Priya','CSE',4,8.10,80.0),
(105,'Neha','ECE',4,8.80,90.5),
(106,'Rohit','ECE',4,6.90,68.0),
(107,'Simran','AIML',4,8.00,74.5),
(108,'Farhan','CSE',4,9.40,95.0),
(109,'Divya','ECE',4,7.60,82.0),
(110,'Karan','AIML',4,8.90,89.0);
```

New this week — extend the dataset with a date column and a related table:

```
-- Add a date column for this week's date-function tasks
ALTER TABLE Student ADD COLUMN Admission_Date DATE;
UPDATE Student SET Admission_Date='2023-08-01' WHERE Student_ID=101;
```

```
UPDATE Student SET Admission_Date='2023-08-01' WHERE Student_ID=102;
UPDATE Student SET Admission_Date='2023-08-02' WHERE Student_ID=103;
UPDATE Student SET Admission_Date='2023-08-02' WHERE Student_ID=104;
UPDATE Student SET Admission_Date='2023-08-03' WHERE Student_ID=105;
UPDATE Student SET Admission_Date='2023-08-03' WHERE Student_ID=106;
UPDATE Student SET Admission_Date='2023-08-01' WHERE Student_ID=107;
UPDATE Student SET Admission_Date='2023-08-02' WHERE Student_ID=108;
UPDATE Student SET Admission_Date='2023-08-03' WHERE Student_ID=109;
UPDATE Student SET Admission_Date='2023-08-01' WHERE Student_ID=110;

-- New table for this week's multi-table / Cartesian product tasks
CREATE TABLE Department (
  Dept_Name VARCHAR(20) PRIMARY KEY,
  HOD       VARCHAR(30),
  Building  VARCHAR(20)
);

INSERT INTO Department VALUES
('AIML','Dr. Mehta','Block C'),
('CSE','Dr. Rao','Block A'),
('ECE','Dr. Iyer','Block B');
```

4. QUESTIONS / TASKS

Q1. Using string functions, display Name in UPPERCASE, its LENGTH, and the first three characters using SUBSTR, for every student.

Q2. Using numeric functions, display CGPA ROUNDed to 1 decimal place, and Attendance TRUNCated to the nearest whole number, for every student.

Q3. Using CONCAT, display a single column combining Name and Department formatted as 'Name (Department)'.

Q4. Using date functions on Admission_Date, display each student's tenure in completed months using TIMESTAMPDIFF(MONTH, Admission_Date, CURDATE()), and the date six months after Admission_Date using DATE_ADD.

Q5. Write a query producing the CARTESIAN PRODUCT (CROSS JOIN) of Student and Department. Note the row count returned and verify it equals (rows in Student) x (rows in Department).

Q6. Write a multi-table query using the old-style (WHERE-based) join to combine Student and Department, matching Student.Department = Department.Dept_Name, displaying Name, Department, Building and HOD.

Q7. Extend the query from the previous task with an additional WHERE condition to display only students whose department is in 'Block A'.

Q8. Compare the row counts from the Cartesian product and the filtered join, and explain in 2-3 lines why an unfiltered Cartesian product is rarely useful in practice.

Q9. Using a WHERE-based multi-table query, display each student's name alongside their department's HOD, sorted by Department.

5. STUDY MATERIAL / REFERENCE

- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- String Functions: https://dev.mysql.com/doc/refman/8.0/en/string-functions.html
- Numeric / Mathematical Functions: https://dev.mysql.com/doc/refman/8.0/en/mathematical-functions.html
- Date and Time Functions: https://dev.mysql.com/doc/refman/8.0/en/date-and-time-functions.html
- Joins (Multi-table queries, Cartesian product): https://dev.mysql.com/doc/refman/8.0/en/join.html

6. EVALUATION RUBRIC

Evalu	Marks

ationCriteria	
Correctuseofstring,numericanddatefunctions	3
Corr	2

ectCROSSJOIN/Cartesianproductquery	
CorrectWHERE-basedmulti-ta	2

blejoinwithfilterconditionn	
Correctoutputinterpretationandverifi	1

fication	
SQL script quality and formatting	1
Viva/spot-check	1
TOTAL	**10**

7. EVALUATION PROCESS

5. Submission check — SQL script and outputs received within the deadline.
6. Execution check — queries are executed successfully and outputs are verified.
7. Rubric scoring — marks are awarded against the stated criteria.
8. Viva/spot-check — brief questions may be asked to verify individual understanding.

GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	**Course Code**	E2UC302B
Program	B.Tech.(AIML)	**Academic Year**	2026–27
Semester/Batch	III	**Lab Assignment No**	06

		o.	
Set	SETC	**Date**	
FacultyIn-Charge	Mr.ArunendraManiTripathi	**Max.Marks**	10
WeekNo.	5	**Duration**	90minutes

CourseOutcome(s)	CO3: Apply SQL techniques, including single-row functions and multi-table queries, to retrieve and reformat meaningful information from relational databases.

Skill(s) Addressed	SQL Programming, Single-Row Functions, Multi-table Queries, Healthcare Data Analysis

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Execute all SQL queries in MySQL Workbench (or the prescribed SQL client).
- Use the dataset/schema given below and retain the same table and column names.
- Show both the SQL query and the corresponding output for every question.
- Save the SQL script as RollNo_Name_DBMS_Lab06_SETC.sql.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Use single-row (string, numeric and date) functions to reformat and analyse patient data, then combine Patient with the new Department table using multi-table (Cartesian product and old-style WHERE-based) queries.

3. DATASET / TABLE SCHEMA

Patient table (already created in Week 4 — re-create only if not already present):

```
CREATE TABLE Patient (
  Patient_ID   INT PRIMARY KEY,
  Patient_Name VARCHAR(30),
  Department   VARCHAR(20),
  Age          INT,
  Bill_Amount  DECIMAL(10,2)
);

INSERT INTO Patient VALUES
(1,'Rehan','Cardiology',54,85000),
(2,'Zoya','Pulmonology',29,42000),
(3,'Ishaan','Cardiology',62,95000),
(4,'Tanya','General',45,25000),
(5,'Aryan','Cardiology',70,110000),
(6,'Kavya','Pulmonology',38,38000),
(7,'Suresh','General',58,31000),
(8,'Meera','Pulmonology',33,47000),
(9,'Kabir','Cardiology',67,102000),
(10,'Riya','General',49,28000);
```

New this week — extend the dataset with a date column and a related table:

```
-- Add a date column for this week's date-function tasks
ALTER TABLE Patient ADD COLUMN Admission_Date DATE;
UPDATE Patient SET Admission_Date='2026-01-10' WHERE Patient_ID=1;
UPDATE Patient SET Admission_Date='2026-01-12' WHERE Patient_ID=2;
```

```
UPDATE Patient SET Admission_Date='2026-01-15' WHERE Patient_ID=3;
UPDATE Patient SET Admission_Date='2026-01-18' WHERE Patient_ID=4;
UPDATE Patient SET Admission_Date='2026-01-20' WHERE Patient_ID=5;
UPDATE Patient SET Admission_Date='2026-01-22' WHERE Patient_ID=6;
UPDATE Patient SET Admission_Date='2026-01-25' WHERE Patient_ID=7;
UPDATE Patient SET Admission_Date='2026-01-27' WHERE Patient_ID=8;
UPDATE Patient SET Admission_Date='2026-01-29' WHERE Patient_ID=9;
UPDATE Patient SET Admission_Date='2026-01-30' WHERE Patient_ID=10;

-- New table for this week's multi-table / Cartesian product tasks
CREATE TABLE Department (
  Department VARCHAR(20) PRIMARY KEY,
  Building     VARCHAR(20),
  Head_Doctor  VARCHAR(30)
);

INSERT INTO Department VALUES
('Cardiology','Block D','Dr. Kapoor'),
('Pulmonology','Block E','Dr. Nair'),
('General','Block F','Dr. Sharma');
```

4. QUESTIONS / TASKS

Q1. Using string functions, display Patient_Name in UPPERCASE, its LENGTH, and the first three characters using SUBSTR, for every patient.

Q2. Using numeric functions, display Age ROUNDed to the nearest 10, and the remainder of Age MOD 10, for every patient.

Q3. Using CONCAT, display a single column combining Patient_Name and Department formatted as 'Patient_Name (Department)'.

Q4. Using date functions on Admission_Date, display the number of days elapsed since admission using DATEDIFF(CURDATE(), Admission_Date), and the date ten days after Admission_Date using DATE_ADD.

Q5. Write a query producing the CARTESIAN PRODUCT (CROSS JOIN) of Patient and Department. Note the row count returned and verify it equals (rows in Patient) x (rows in Department).

Q6. Write a multi-table query using the old-style (WHERE-based) join to combine Patient and Department, matching Patient.Department = Department.Department, displaying Patient_Name, Department, Building and Head_Doctor.

Q7. Extend the query from the previous task with an additional WHERE condition to display only patients whose department is located in 'Block D'.

Q8. Compare the row counts from the Cartesian product and the filtered join, and explain in 2-3 lines why an unfiltered Cartesian product is rarely useful in practice.

Q9. Using a WHERE-based multi-table query, display each patient's name alongside their department's Head_Doctor, sorted by Department.

5. STUDY MATERIAL / REFERENCE

- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- String Functions: https://dev.mysql.com/doc/refman/8.0/en/string-functions.html
- Numeric / Mathematical Functions: https://dev.mysql.com/doc/refman/8.0/en/mathematical-functions.html
- Date and Time Functions: https://dev.mysql.com/doc/refman/8.0/en/date-and-time-functions.html
- Joins (Multi-table queries, Cartesian product): https://dev.mysql.com/doc/refman/8.0/en/join.html

6. EVALUATION RUBRIC

Evalua	Marks

tionCriteria	
Correctuseofstring,numericanddatefunctions	3
Corre	2

ctCROSSJOIN/Cartesianproductquery	
CorrectWHERE-basedmulti-tab	2

Ie join with filter condition	
Correct output interpretation and verif	1

ication	
SQL script quality and formatting	1
Viva/spot-check	1
TOTAL	**10**

7. EVALUATION PROCESS

9. Submission check — SQL script and outputs received within the deadline.
10. Execution check — queries are executed successfully and outputs are verified.
11. Rubric scoring — marks are awarded against the stated criteria.
12. Viva/spot-check — brief questions may be asked to verify individual understanding.

QUESTION-WISE MAPPING & SOLUTION — SET A

A. Question-wise Mapping

Question	BTL	SOLO Level	CO
Q1	L3 – Apply	Multistructural	CO3
Q2	L3 – Apply	Relational	CO3
Q3	L3 – Apply	Relational	CO3
Q4	L3 – Apply	Relation	CO3

		al	
Q5	L3–Apply	Relational	CO3
Q6	L3–Apply	Relational	CO3
Q7	L4–Analyze	Relational	CO3
Q8	L4–Analyze	Relational	CO3
Q9	L4–Analyze	Relational	CO3

B. Solution

Execute the following MySQL queries using the Employee / Department dataset in the assignment.

Q1.

```
SELECT UPPER(Emp_Name) AS Name_Upper, LENGTH(Emp_Name) AS Name_Length, SUBSTR(Emp_Name,1,3) AS First3
FROM Employee;
```

Q2.

```
SELECT Emp_Name, ROUND(Salary,-3) AS Salary_Rounded, TRUNCATE(Salary,-3) AS Salary_Truncated,
MOD(Salary,1000) AS Salary_Remainder
FROM Employee;
```

Q3.

```
SELECT CONCAT(Emp_Name,' (',Department,')') AS Emp_Dept
FROM Employee;
```

Q4.

```
SELECT Emp_Name, TIMESTAMPDIFF(MONTH, Joining_Date, CURDATE()) AS Tenure_Months,
DATE_ADD(Joining_Date, INTERVAL 1 YEAR) AS Date_After_1Yr
FROM Employee;
```

Q5.

```
SELECT * FROM Employee CROSS JOIN Department;
-- Expected row count: 10 (Employee) x 3 (Department) = 30
```

Q6.

```
SELECT E.Emp_Name, E.Department, D.Location, D.HOD
FROM Employee E, Department D
WHERE E.Department = D.Dept_Name;
```

Q7.

```
SELECT E.Emp_Name, E.Department, D.Location, D.HOD
FROM Employee E, Department D
WHERE E.Department = D.Dept_Name AND D.Location = 'Bengaluru';
```

Q8.

```
-- Cartesian product row count = 30 (all Employee-Department combinations, most meaningless)
-- Filtered join row count = 10 (only real Employee-Department matches)
-- Explanation: a Cartesian product pairs every row of one table with every row of the other
regardless of
-- relevance, so most rows are meaningless; a filtered/equi-join keeps only the pairs that correctly
-- correspond to real relationships, which is why WHERE/JOIN conditions are used in practice.
```

Q9.

```
SELECT E.Emp_Name, E.Department, D.HOD
FROM Employee E, Department D
WHERE E.Department = D.Dept_Name
ORDER BY E.Department;
```

Verification: *Compare the output with the supplied dataset and confirm that CROSS JOIN returns (rows in Employee) x (rows in Department) total rows, while the WHERE-based join returns only the rows with matching Department values.*

QUESTION-WISE MAPPING & SOLUTION — SET B

A. Question-wise Mapping

Question	BTL	SOLO Level	CO
Q1	L3 – Apply	Multistructural	CO3
Q2	L3 – Apply	Relational	CO3
Q3	L3 – Apply	Relational	CO3
Q4	L3 – Apply	Relation	CO3

		al	
Q5	L3–Apply	Relational	CO3
Q6	L3–Apply	Relational	CO3
Q7	L4–Analyze	Relational	CO3
Q8	L4–Analyze	Relational	CO3
Q9	L4–Analyze	Relational	CO3

B. Solution

Execute the following MySQL queries using the Student / Department dataset in the assignment.

Q1.

```
SELECT UPPER(Name) AS Name_Upper, LENGTH(Name) AS Name_Length, SUBSTR(Name,1,3) AS First3
FROM Student;
```

Q2.

```
SELECT Name, ROUND(CGPA,1) AS CGPA_Rounded, TRUNCATE(Attendance,0) AS Attendance_Truncated
FROM Student;
```

Q3.

```
SELECT CONCAT(Name,' (',Department,')') AS Student_Dept
FROM Student;
```

Q4.

```
SELECT Name, TIMESTAMPDIFF(MONTH, Admission_Date, CURDATE()) AS Tenure_Months,
DATE_ADD(Admission_Date, INTERVAL 6 MONTH) AS Date_After_6Mo
FROM Student;
```

Q5.

```
SELECT * FROM Student CROSS JOIN Department;
-- Expected row count: 10 (Student) x 3 (Department) = 30
```

Q6.

```
SELECT S.Name, S.Department, D.Building, D.HOD
FROM Student S, Department D
WHERE S.Department = D.Dept_Name;
```

Q7.

```
SELECT S.Name, S.Department, D.Building, D.HOD
FROM Student S, Department D
WHERE S.Department = D.Dept_Name AND D.Building = 'Block A';
```

Q8.

```
-- Cartesian product row count = 30 (all Student-Department combinations, most meaningless)
-- Filtered join row count = 10 (only real Student-Department matches)
-- Explanation: a Cartesian product pairs every row of one table with every row of the other
regardless of
-- relevance, so most rows are meaningless; a filtered/equi-join keeps only the pairs that correctly
-- correspond to real relationships, which is why WHERE/JOIN conditions are used in practice.
```

Q9.

```
SELECT S.Name, S.Department, D.HOD
FROM Student S, Department D
WHERE S.Department = D.Dept_Name
ORDER BY S.Department;
```

Verification: *Compare the output with the supplied dataset and confirm that CROSS JOIN returns (rows in Student) x (rows in Department) total rows, while the WHERE-based join returns only the rows with matching Department values.*

QUESTION-WISE MAPPING & SOLUTION — SET C

A. Question-wise Mapping

Question	BTL	SOLO Level	CO
Q1	L3 – Apply	Multistructural	CO3
Q2	L3 – Apply	Relational	CO3
Q3	L3 – Apply	Relational	CO3
Q4	L3 – Apply	Relation	CO3

		al	
Q5	L3–Apply	Relational	CO3
Q6	L3–Apply	Relational	CO3
Q7	L4–Analyze	Relational	CO3
Q8	L4–Analyze	Relational	CO3
Q9	L4–Analyze	Relational	CO3

B. Solution

Execute the following MySQL queries using the Patient / Department dataset in the assignment.

Q1.

```
SELECT UPPER(Patient_Name) AS Name_Upper, LENGTH(Patient_Name) AS Name_Length,
SUBSTR(Patient_Name,1,3) AS First3
FROM Patient;
```

Q2.

```
SELECT Patient_Name, ROUND(Age,-1) AS Age_Rounded, MOD(Age,10) AS Age_Remainder
FROM Patient;
```

Q3.

```
SELECT CONCAT(Patient_Name,' (',Department,')') AS Patient_Dept
FROM Patient;
```

Q4.

```
SELECT Patient_Name, DATEDIFF(CURDATE(), Admission_Date) AS Days_Since_Admission,
DATE_ADD(Admission_Date, INTERVAL 10 DAY) AS Date_After_10Days
FROM Patient;
```

Q5.

```
SELECT * FROM Patient CROSS JOIN Department;
-- Expected row count: 10 (Patient) x 3 (Department) = 30
```

Q6.

```
SELECT P.Patient_Name, P.Department, D.Building, D.Head_Doctor
FROM Patient P, Department D
WHERE P.Department = D.Department;
```

Q7.

```
SELECT P.Patient_Name, P.Department, D.Building, D.Head_Doctor
FROM Patient P, Department D
WHERE P.Department = D.Department AND D.Building = 'Block D';
```

Q8.

```
-- Cartesian product row count = 30 (all Patient-Department combinations, most meaningless)
-- Filtered join row count = 10 (only real Patient-Department matches)
-- Explanation: a Cartesian product pairs every row of one table with every row of the other
regardless of
-- relevance, so most rows are meaningless; a filtered/equi-join keeps only the pairs that correctly
-- correspond to real relationships, which is why WHERE/JOIN conditions are used in practice.
```

Q9.

```
SELECT P.Patient_Name, P.Department, D.Head_Doctor
FROM Patient P, Department D
WHERE P.Department = D.Department
ORDER BY P.Department;
```

Verification: *Compare the output with the supplied dataset and confirm that CROSS JOIN returns (rows in Patient) x (rows in Department) total rows, while the WHERE-based join returns only the rows with matching Department values.*